Universidad Nacional de Ingeniería

Escuela Profesional de Ingeniería de Software – Facultad de Ingeniería Industrial y de Sistemas

SW505 – Contrucción de Software I

Laboratorio: Pruebas unitarias

NOTA: Subir el código implementado y los reportes a su repositorio de **Github** del curso antes de 20:00 horas.
Tiempo de desarrollo: 60 minutos

1. Fundamento

Se define la prueba unitaria como el nivel más bajo de prueba, donde se verifica el comportamiento de componentes individuales (funciones, métodos, clases) de forma aislada . El estándar enfatiza que las pruebas deben ser **planificadas, especificadas, ejecutadas y registradas**. Por ello, esta guía no solo exige que las pruebas pasen, sino que el alumno demuestre que sabe **documentar casos de prueba** (entrada, precondición, resultado esperado, resultado actual, estado) y **generar reportes formales** de ejecución.

2. Objetivos

Al finalizar el laboratorio, el alumno será capaz de:

1. Implementar pruebas unitarias en Python usando pytest para los **tipos fundamentales** de prueba unitaria.
2. Documentar cada caso de prueba siguiendo una plantilla estructurada.
3. Generar **reportes de ejecución** en formato HTML y **reportes de cobertura**.
4. Interpretar los resultados para identificar brechas en la calidad del código.

3. Código Bajo Prueba

Cree la siguiente estructura de proyecto:

```
lab_pytest/
├── calculadora.py
└── tests/
    └── test_calculadora.py
```

Archivo `calculadora.py`:

```
# calculadora.py

def sumar(a, b):
    """Retorna la suma de dos números."""
    return a + b

def dividir(a, b):
    """Retorna la división de a entre b. Lanza ValueError si b es cero."""
    if b == 0:
        raise ValueError("No se puede dividir por cero")
    return a / b

def es_mayor_de_edad(edad):
    """Retorna True si edad >= 18, False en caso contrario.
    Lanza ValueError si edad es negativa."""
    if edad < 0:
        raise ValueError("La edad no puede ser negativa")
    return edad >= 18
```

4. Tipos de Prueba Unitaria a Implementar

El alumno debe escribir **al menos un caso por cada tipo** en `tests/test_calculadora.py`.

4.1 Prueba de Camino Normal (Happy Path)

Verifica que la función hace lo correcto con entradas válidas típicas.

Plantilla de documentación del caso de prueba (obligatoria para cada caso):

Campo	Valor
ID	CP-001
Tipo	Camino Normal
Descripción	Sumar dos enteros positivos
Precondición	Ninguna
Datos de entrada	a=2, b=3
Resultado esperado	5
Resultado actual	*(se llena al ejecutar)*
Estado	*(Aprobado / Reprobado)*

Implementación:

```
def test_sumar_positivos():
    assert sumar(2, 3) == 5
```

4.2 Prueba de Valor Límite (Boundary)

Prueba valores en los bordes de las condiciones. Para `es_mayor_de_edad`, los límites son 17 y 18.

Campo	Valor
ID	CP-002
Tipo	Valor Límite
Descripción	Verificar edad exactamente en el límite (18)
Precondición	Ninguna
Datos de entrada	edad=18
Resultado esperado	True
Resultado actual	
Estado	

Implementación:

```
def test_mayor_edad_limite_inferior():
    assert es_mayor_de_edad(18) is True

def test_menor_edad_limite_superior():
    assert es_mayor_de_edad(17) is False
```

4.3 Prueba de Manejo de Excepciones

Verifica que la función lanza la excepción correcta ante entradas inválidas.

Campo	Valor
ID	CP-003
Tipo	Excepción
Descripción	Dividir por cero debe lanzar ValueError
Precondición	Ninguna
Datos de entrada	a=10, b=0
Resultado esperado	ValueError con mensaje "No se puede dividir por cero"
Resultado actual	
Estado	

Implementación:

```
import pytest

def test_dividir_por_cero():
    with pytest.raises(ValueError, match="No se puede dividir por cero"):
        dividir(10, 0)
```

4.4 Prueba de Entrada Inválida (Negativa)

Verifica el rechazo de datos fuera del dominio válido.

Campo	Valor
ID	CP-004
Tipo	Entrada Inválida
Descripción	Edad negativa debe lanzar ValueError
Precondición	Ninguna
Datos de entrada	edad=-5
Resultado esperado	ValueError con mensaje "La edad no puede ser negativa"
Resultado actual	
Estado	

Implementación:

```
def test_edad_negativa():
    with pytest.raises(ValueError, match="La edad no puede ser negativa"):
        es_mayor_de_edad(-5)
```

4.5 Prueba con Múltiples Aserciones (Parametrizada)

Para eficiencia, pytest permite parametrizar múltiples casos en una sola función.

Campo	Valor
ID	CP-005
Tipo	Parametrizada (múltiples entradas)
Descripción	Sumar varios pares de números
Precondición	Ninguna
Datos de entrada	(1,1)→2; (-1,1)→0; (0,0)→0; (100,200)→300
Resultado esperado	Los valores indicados
Resultado actual	
Estado	

Implementación:

```
@pytest.mark.parametrize("a, b, esperado", [
    (1, 1, 2),
    (-1, 1, 0),
    (0, 0, 0),
    (100, 200, 300),
])
def test_sumar_parametrizado(a, b, esperado):
    assert sumar(a, b) == esperado
```

5. Ejecución y Generación de Reportes

5.1 Instalación de dependencias

```
pip install pytest pytest-html pytest-cov
```

5.2 Ejecución básica con reporte HTML

Desde la raíz del proyecto:

```
pytest tests/ --html=reporte_ejecucion.html --self-contained-html
```

Esto genera un archivo `reporte_ejecucion.html` que puede abrir en el navegador. El reporte muestra: nombre de cada prueba, estado (PASSED/FAILED), duración y, en caso de fallo, la traza del error .

5.3 Ejecución con reporte de cobertura

```
pytest tests/ --cov=calculadora --cov-report=html --cov-report=term-missing
```

Salida esperada en terminal (ejemplo aproximado):

text

```
Name              Stmts   Miss  Cover   Missing
-------------------------------------------------
calculadora.py       12      0   100%
-------------------------------------------------
TOTAL                12      0   100%
```

El reporte HTML de cobertura se genera en `htmlcov/index.html`. Las líneas **verdes** están cubiertas; las **rojas**, no .

5.4 Reporte combinado (ejecución + cobertura)

```
pytest tests/ --html=reporte_completo.html --self-contained-html --
cov=calculadora --cov-report=html
```

6. Entregables del Alumno

El alumno debe subir a la plataforma:

1. **`test_calculadora.py`** con los 5 tipos de prueba implementados.
2. **Tabla de documentación de casos** (la plantilla del punto 4) completada con los resultados reales obtenidos tras la ejecución.
3. **`reporte_completo.html`** generado por pytest-html.
4. **Captura o archivo** de la carpeta `htmlcov/index.html` mostrando el porcentaje de cobertura.
5. **Respuesta breve** (máximo 5 líneas): ¿Qué línea de `calculadora.py` quedaría sin cubrir si se elimina el test de edad negativa? Justifique usando el reporte de cobertura.